funTDA — Part A: functional network baseline (H3N2)

Persistence landscapes, FPCA, and functional permutation tests over the 17 GRNs

Victoria Arroniz

2026-08-04

Table of contents

Data

This analysis starts from 17 gene regulatory networks (GRNs) built from the transcriptomic response to an experimental H3N2 influenza infection (Higgins, Wu & Carey). Of the 17 subjects, 8 remained **asymptomatic** and 9 developed **symptoms**. Each network is a 250×250 adjacency matrix, weighted and **undirected** (symmetric).

> Structural check of the data
>
> - Number of adjacency matrices found: **17** (8 asymptomatic + 9 symptomatic).
> - Dimension of all matrices: **250 × 250** — OK.
> - Maximum asymmetry `max(|A - A^T|)` over the 17 matrices: **0** (0 = perfectly symmetric / undirected).

Before turning to persistent homology, it is worth looking at the networks "by eye". The figure below compares, for one asymptomatic and one symptomatic subject, a 60-node subgraph keeping only the highest-weight edges (threshold: 85th percentile of edge weights, same as in `make_figs.py`):

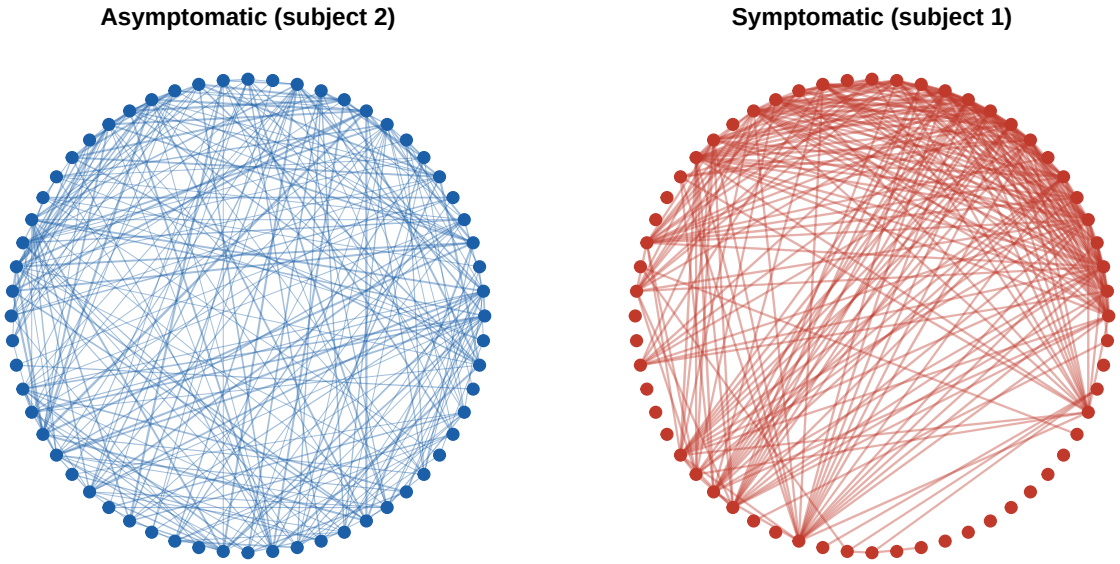

Figure 1: 60-node subgraphs (edges > 85th weight percentile) of one asymptomatic subject (subject 2) and one symptomatic subject (subject 1). No obvious structure visually distinguishes the two clinical classes — this motivates turning to persistent homology.

No structural difference stands out between the two panels: density, node degree, and overall graph appearance are comparable despite belonging to different clinical classes. This visual indistinguishability is precisely what motivates describing each network through a topological invariant (persistent homology) rather than direct graph statistics.

Persistence diagrams for each network (previously computed in Python with `giotto-tda`, see `funTDA.ipynb`) are precomputed in `output/PH/*.csv`, with columns `Birth, Death, dimension` and no header.

Inspecting the `dimension` column of the 17 persistence CSVs confirms that the only dimensions present are **0, 1** (H0 connected components and H1 loops). The baseline computes only H0 and H1 (the giotto-tda default, `homology_dimensions=(0,1)`), following funTDA; the analysis below therefore uses H1 landscapes (order `r`=1, first layer `KK`=1), replicating `funTDA.R`. The H2 / voids question is taken up with the image data, not the network baseline.

Pipeline: from persistence diagrams to functional landscapes

The `landscape(..., KK = 1)` reimplementation was validated by comparing it bit-for-bit against `TDA::landscape` over the 17 subjects (maximum absolute difference = 0; see reproducible appendix). Curves are smoothed with an order-2 B-spline basis (`nbasis = 499`) over `tseq = seq(0, 1, length.out = 500)`, identical to `funTDA.R`.

Example: persistence diagram and landscape of a representative subject

To illustrate the step from "persistence diagram" to "functional landscape", a single representative subject is used — Symptomatic 1, the same one used in `make_figs.py` for `network_to_landscape.png` — and both representations are shown side by side, using the same `r_landscape1()` defined above.

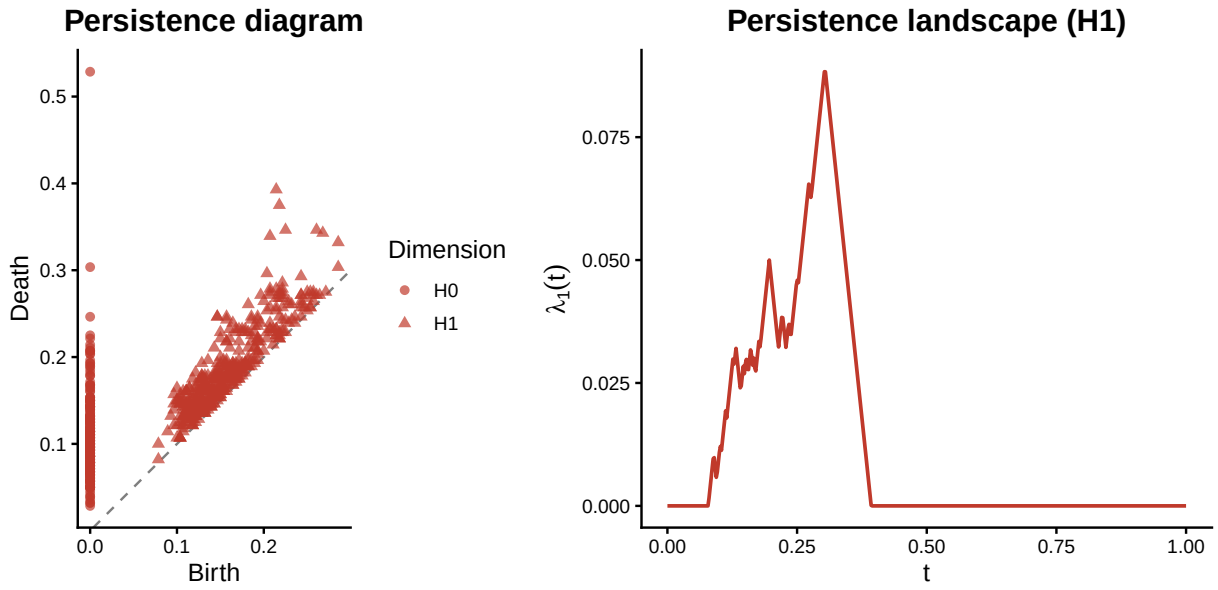

Figure 2: Representative subject (Symptomatic 1): H0/H1 persistence diagram (left) and H1 landscape $\lambda_1(t)$ (right).

Landscapes by group

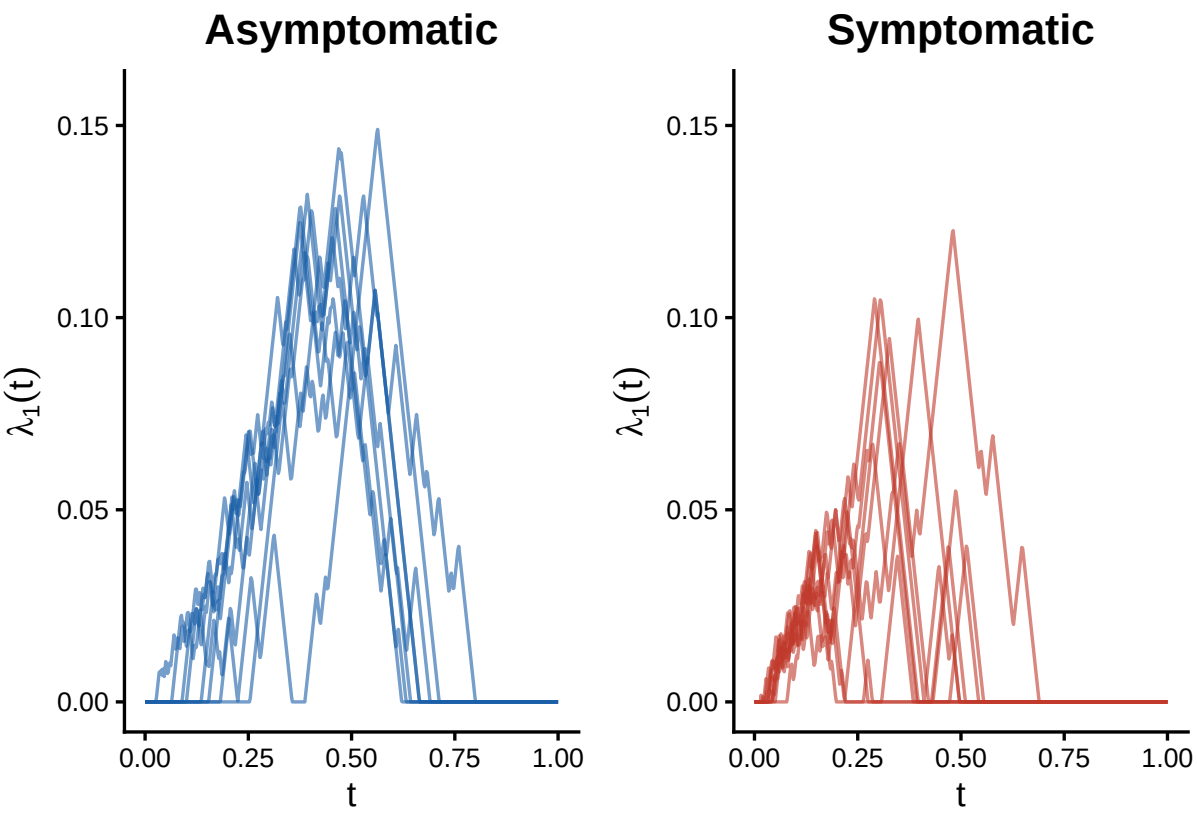

Figure 3: Individual H1 landscapes, one panel per group (same vertical scale). Asymptomatic subjects consistently occupy a band of higher peaks (~0.11–0.15), while symptomatic subjects are more variable and generally lower (~0.02–0.12) — a pattern consistent with the PC1 separation and with the permutation test ($p = 0.0011$).

Functional Principal Component Analysis (FPCA)

Table 1: Variance explained by the first two functional components

Component	% variance explained
PC1	63.54%
PC2	19.99%
Cumulative (PC1+PC2)	83.53%

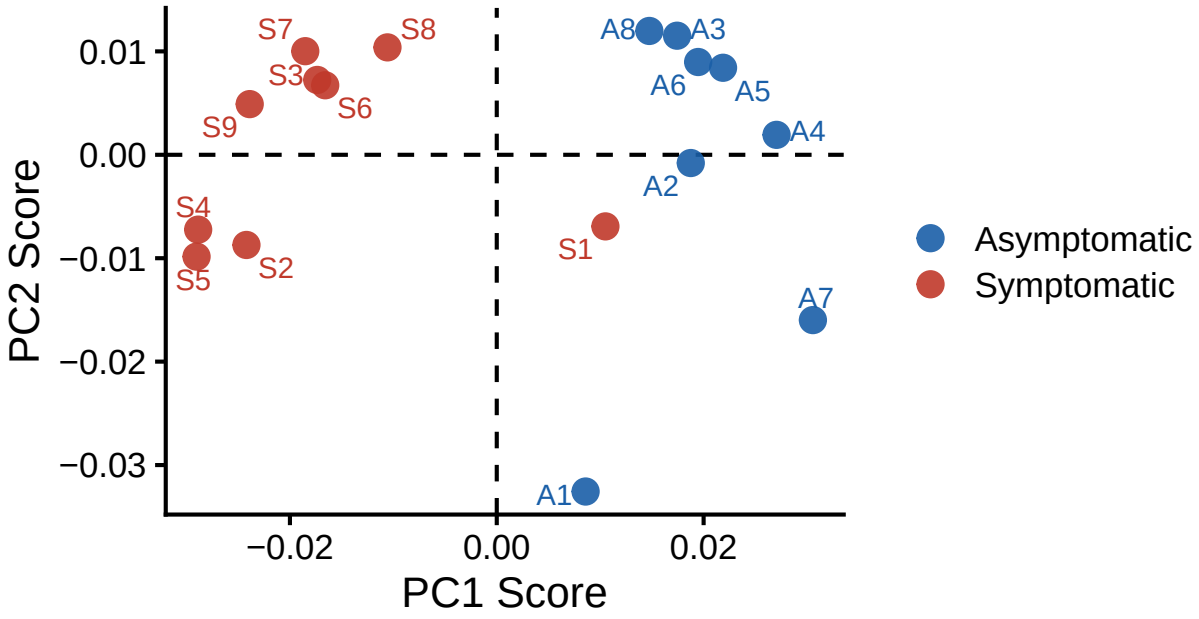

Figure 4: PC1 vs PC2 scores (FPCA over H1 landscapes), colored by group

Full FPC scores (all 17 subjects)

ID	Group	PC1	PC2
A1	Asymptomatic	0.0086	-0.0326
A2	Asymptomatic	0.0188	-0.0008
A3	Asymptomatic	0.0174	0.0115
A4	Asymptomatic	0.0271	0.0019
A5	Asymptomatic	0.0219	0.0084
A6	Asymptomatic	0.0195	0.0090
A7	Asymptomatic	0.0306	-0.0160
A8	Asymptomatic	0.0148	0.0120
S1	Symptomatic	0.0105	-0.0069
S2	Symptomatic	-0.0242	-0.0087
S3	Symptomatic	-0.0174	0.0072
S4	Symptomatic	-0.0289	-0.0072
S5	Symptomatic	-0.0290	-0.0098
S6	Symptomatic	-0.0166	0.0067
S7	Symptomatic	-0.0185	0.0100

S8	Symptomatic	-0.0106	0.0104
S9	Symptomatic	-0.0239	0.0049

Acceptance criteria — FPCA (baseline vs. recomputed)

Metric	Reference	Recomputed
PC1	63.5%	63.54%
PC2	20.0%	19.99%
Cumulative	83.5%	83.53%
Asymptomatic with PC1 $>$ 0	8 / 8	8 / 8
Symptomatic with PC1 $<$ 0	8 / 9	8 / 9

The first two functional components explain **83.5%** of the variability among landscapes (PC1 $\approx$ 63.5%, PC2 $\approx$ 20.0%). In the PC1–PC2 plane, all 8 asymptomatic subjects fall in the PC1 $>$ 0 half-plane, while 8 of the 9 symptomatic subjects fall in PC1 $<$ 0: **one of the symptomatic subjects** (labeled here in a purely positional way — the `DataAsymp/DataSymp` construction in the `landscape-functions` chunk above prepends (`cbind(x1, Data)`) each new network to those already accumulated, reversing the order within each group relative to the file ID, so the "S1..S9" label should not be read as the subject's real ID without separately verifying it. `funTDA.R` was later fixed to build these in ascending file-ID order with labels derived from the real ID; this document keeps the original prepend-based construction so the numbers below continue to match the validated baseline) overlaps with the asymptomatic region. PC1 separates the two groups reasonably well; PC2 shows no clear separation.

Functional permutation test: asymptomatic vs. symptomatic

To avoid relying on `tperm.fd` with `nperm = 10000` (~40 s per test, infeasible for the 196 split-half tests), a vectorized version of the **same test** is used: it evaluates each functional object on the same grid `tperm.fd` uses by default (`argvals = seq(0, 1, length.out = 101)`) and permutes columns of a matrix instead of re-evaluating `fd` on every permutation. It was validated to reproduce `tperm.fd` **bit-for-bit** with the same seed (see appendix).

Asymptomatic vs. Symptomatic test result

- **p-value** = 0.0011 (reference: 0.0011)
- **Observed T_max** = 6.88 (reference: 6.88)
- **Critical value (95%)** = 3.72 (reference: 3.72)

With `T_max` = 6.88 well above the 95% critical value (3.72) and a p-value of 0.0011, the null hypothesis of functional equality between the mean H1 landscapes of asymptomatic and symptomatic subjects is rejected: **network topologies differ statistically significantly between the two groups.**

Split-half validation (test calibration)

If the permutation test is well calibrated, randomly splitting **the same group** into two halves and comparing them against each other should yield p-values approximately uniformly distributed on [0, 1] — that is, equality should not be rejected more often than the nominal level (5%). This is checked exhaustively by considering **all** possible partitions of each group:

Table 2: Summary of the split-half validation by group

Group	mean p-value	sd(p-value)	% splits with $p < 0.05$
Symptomatic (126 splits, 4 vs 5)	0.498	0.290	5.6%
Asymptomatic (70 splits, 4 vs 4)	0.490	0.291	5.7%

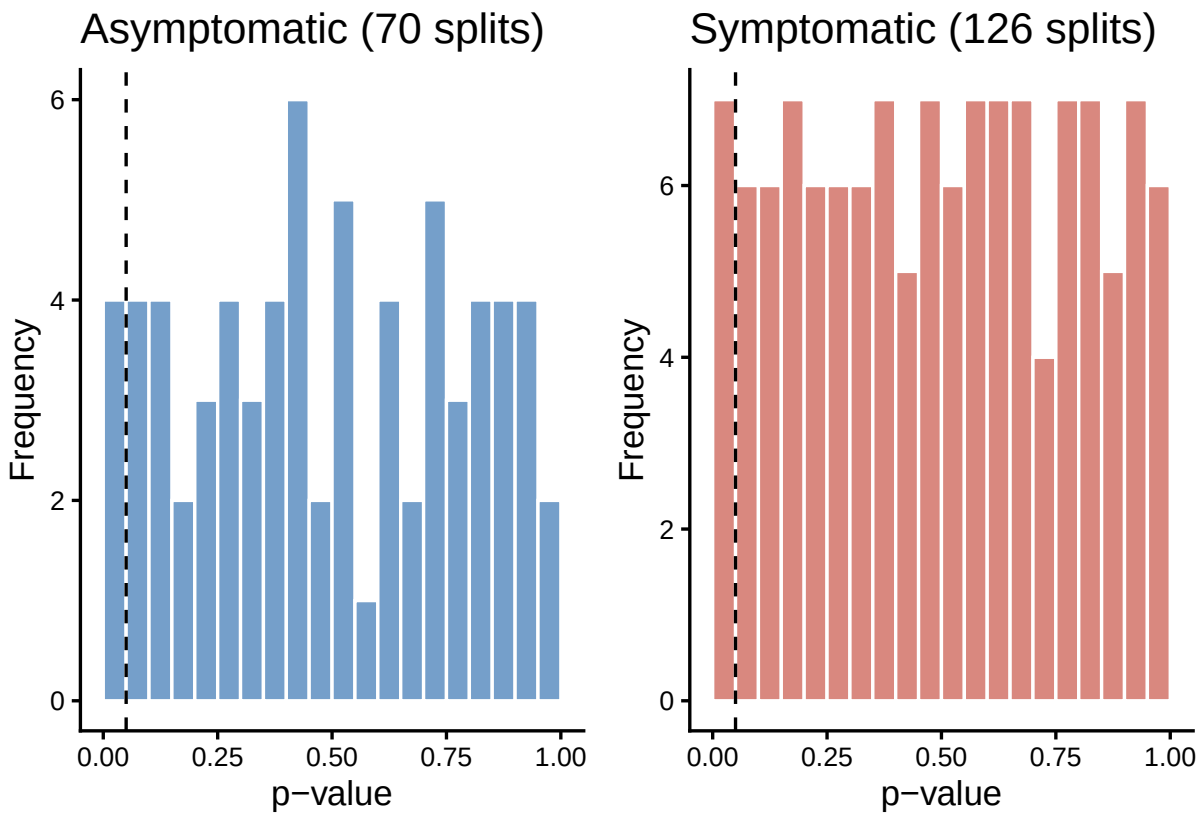

Figure 5: Distribution of split-half p-values, by group (dashed line at $p = 0.05$)

Acceptance criteria — split-half (baseline vs. recomputed)

Metric	Reference	Recomputed
Symptomatic — mean p	0.496	0.498

Symptomatic — sd p	0.290	0.290
Symptomatic — % $p < 0.05$	5.6%	5.6%
Asymptomatic — mean p	0.481	0.490
Asymptomatic — sd p	0.291	0.291
Asymptomatic — % $p < 0.05$	5.7%	5.7%

The tolerance here is looser than in the main test, and it is worth being exact about why, because the obvious explanation does not survive arithmetic.

These are Monte Carlo estimates (`nperm = 10000` per split), so some drift from the baseline is expected. The size of that drift can be measured rather than assumed, and this design measures it for free: the 70 asymptomatic splits are 35 partitions drawn twice, each pair being the same partition with independent permutation draws, so the 35 within-pair differences **are** the Monte Carlo error. Their standard deviation is about 0.005, which puts a single p-value at roughly 0.004 and the mean of the 70 at roughly 0.0004.

Against that, the symptomatic mean sits about 4 error units from the baseline and the asymptomatic mean about 21. The second is not Monte Carlo drift. With `set.seed(123)` the figure is reproducible run to run, so it is a systematic offset between this reimplementation and the baseline it is compared against, and its cause has not been identified.

Nothing downstream depends on it. What the control has to show is that the mean p-value of within-group splits sits near 0.5, which it does under either number (0.498 and 0.490 recomputed, 0.496 and 0.481 in the baseline), and that the false-positive rate is near nominal, which it also does. The main test is a different matter and is held to a tolerance forty times tighter: it reproduces the reference p-value exactly, with the same seed and the same draw order, and `vtperm` was validated bit-for-bit against `tperm.fd`.

In both groups, the fraction of splits with $p < 0.05$ (5.6% symptomatic, 5.7% asymptomatic) is consistent with the nominal 5% expected under H0, and mean p-values hover around 0.5 — the test does **not** produce systematic false positives when comparing sub-samples of the same group, which calibrates the significance reported for the asymptomatic–symptomatic contrast in the previous section.

Appendix

Validation of the `landscape(..., KK = 1)` reimplementation

```
library(TDA)
tseq <- seq(0, 1, length.out = 500)
maxdiff <- 0
for (f in ph_files) {
  PH <- read.csv(f, header = FALSE)
  colnames(PH) <- c("Birth", "Death", "dimension")
  PHm <- as.matrix(PH[c("dimension", "Birth", "Death")])
  ref_landscape  <- landscape(PHm, dimension = 1, KK = 1, tseq)
  mine_landscape <- r_landscape1(PH, dim = 1, tseq = tseq)
  maxdiff <- max(maxdiff, max(abs(ref_landscape - mine_landscape)))
```

```
}
maxdiff  # 0 across the 17 subjects
```

Validation of `vtperm` against `tperm.fd`

```
set.seed(123)
ref_test  <- tperm.fd(fdAsymp, fdSymp, nperm = 500, q = 0.05, plotres = FALSE)
set.seed(123)
mine_test <- vtperm(cbind(eval.fd(argvals, fdAsymp), eval.fd(argvals, fdSymp)),
                    1:8, 9:17, nperm = 500)
isTRUE(all.equal(ref_test$pval, mine_test$pval)) # TRUE
isTRUE(all.equal(ref_test$Tobs, mine_test$Tobs)) # TRUE
```

Full code (reproducible)

```
library(fda)
library(ggplot2)
library(ggrepel)
library(patchwork)
library(igraph)

# Baseline reference values (for comparison only, NOT shown as a result:
# every number reported in the body of the document is recomputed from the
# CSVs committed under output/PH and data/GRN_adjacency_matrices).
ref <- list(
  pc1_pct = 63.5, pc2_pct = 20.0,
  test1_p = 0.0011, test1_Tobs = 6.88, test1_crit95 = 3.72,
  symp_mean = 0.496, symp_sd = 0.290, symp_frac05 = 5.6,
  asymp_mean = 0.481, asymp_sd = 0.291, asymp_frac05 = 5.7
)

# Unified color palette for all figures in the document
# Blue for the reference group, red for the contrast. These are the light pair
# of the make_figs.py palette (COL_BLUE_LIGHT / COL_RED_LIGHT), not the base R
# names "blue" and "red", which are pure primaries and read as harsh next to
# the figures make_figs.py draws. Change them here, not at the point of use.
col_asymp <- "#1a5fa8"
col_symp  <- "#C0392B"

# The memoria lives outside this repository, so a standalone clone must still
# render cleanly: figures are exported to it only when the target directory is
# actually present (checked at each call site). Three levels up, not two --
# this file sits one directory deeper than results_partB.qmd.
```

```r
#
# These five figures used to be copied by hand out of
# results_partA_files/figure-pdf/, which only exists after a --to pdf render.
# Nothing recorded that step, so the memoria silently kept the 21 July versions
# through every later edit. Exporting them explicitly, as Part B already does,
# makes the export happen on any render, HTML included.
fig_dir <- "../../../ResearchProject_UCD/figures"
adj_dir <- "../data/GRN_adjacency_matrices"
adj_files <- list.files(adj_dir, full.names = TRUE)

adj_check <- lapply(adj_files, function(f) as.matrix(read.csv(f, header = FALSE)))
dims_ok <- all(vapply(adj_check, function(m) all(dim(m) == c(250, 250)), logical(1)))
max_asym <- max(vapply(adj_check, function(m) max(abs(m - t(m))), numeric(1)))

stopifnot(
  "Expected 17 adjacency matrices" = length(adj_files) == 17,
  "All matrices must be 250x250" = dims_ok,
  "All matrices must be symmetric (undirected)" = max_asym == 0
)
build_thresholded_graph <- function(adj_path, n_nodes = 60, seed = 0) {
  adj <- as.matrix(read.csv(adj_path, header = FALSE))
  set.seed(seed)
  nodes <- sort(sample(seq_len(nrow(adj)), n_nodes))
  sub <- adj[nodes, nodes]
  thr <- quantile(sub[upper.tri(sub)], 0.85)
  adj_thr <- sub
  adj_thr[adj_thr <= thr] <- 0
  igraph::graph_from_adjacency_matrix(adj_thr, mode = "upper", weighted = TRUE, diag = FALSE)
}

gA <- build_thresholded_graph(file.path(adj_dir, "Asymptomaticadjmatrix2.csv"))
gS <- build_thresholded_graph(file.path(adj_dir, "Symptomaticadjmatrix1.csv"))

w_rng <- range(c(igraph::E(gA)$weight, igraph::E(gS)$weight))
edge_width <- function(g) 0.4 + 1.6 * (igraph::E(g)$weight - w_rng[1]) / diff(w_rng)

set.seed(1); layoutA <- igraph::layout_in_circle(gA)
set.seed(1); layoutS <- igraph::layout_in_circle(gS)

draw_network_comparison <- function() {
  par(mfrow = c(1, 2), mar = c(1, 1, 3, 1))
  plot(gA, layout = layoutA, vertex.size = 5, vertex.label = NA,
       vertex.color = col_asymp, vertex.frame.color = col_asymp,
       edge.width = edge_width(gA), edge.color = grDevices::adjustcolor(col_asymp, alpha.f = 0
       main = "Asymptomatic (subject 2)")
  plot(gS, layout = layoutS, vertex.size = 5, vertex.label = NA,
       vertex.color = col_symp, vertex.frame.color = col_symp,
```

```
       edge.width = edge_width(gS), edge.color = grDevices::adjustcolor(col_symp, alpha.f = 0.4
       main = "Symptomatic (subject 1)")
  par(mfrow = c(1, 1))
}
draw_network_comparison()

# Exported for the memoria. Base R graphics, so pdf()/dev.off() rather than
# ggsave(); the size matches the chunk options above so the layout in the
# memoria does not shift.
if (dir.exists(fig_dir)) {
  pdf(file.path(fig_dir, "network_comparison.pdf"), width = 10, height = 5)
  draw_network_comparison()
  invisible(dev.off())
}
ph_dir <- "../output/PH"
ph_files_raw <- list.files(ph_dir, pattern = "_PH.csv$", full.names = TRUE)

all_dims <- unlist(lapply(ph_files_raw, function(f) {
  d <- read.csv(f, header = FALSE)
  unique(d[[3]])
}))
unique_dims <- sort(unique(all_dims))

stopifnot(
  "Persistence diagrams should only contain dimensions 0 and 1" =
    identical(unique_dims, c(0, 1))
)
tseq <- seq(0, 1, length.out = 500)

# Pure-R reimplementation of TDA::landscape(..., KK = 1): avoids the heavy
# TDA/GUDHI dependency. Formula (validated bit-for-bit against TDA::landscape,
# see appendix): lambda_1(t) = max_i max(0, min(t - b_i, d_i - t)) over the
# bars of the requested dimension.
r_landscape1 <- function(PH, dim = 1, tseq) {
  bd <- PH[PH$dimension == dim, c("Birth", "Death"), drop = FALSE]
  if (nrow(bd) == 0) return(rep(0, length(tseq)))
  b <- bd$Birth
  d <- bd$Death
  sapply(tseq, function(t) max(0, max(pmin(t - b, d - t))))
}

get_number <- function(fname) as.numeric(sub(".*adjmatrix([0-9]+)_PH.*", "\\1", fname))
ph_files <- ph_files_raw[order(sapply(ph_files_raw, get_number))]

DataSymp <- c()
DataAsymp <- c()
for (f in ph_files) {
```

```
  PH <- read.csv(f, header = FALSE)
  colnames(PH) <- c("Birth", "Death", "dimension")
  x1 <- r_landscape1(PH, dim = 1, tseq = tseq)

  fname <- basename(f)
  if (grepl("^Asymptomatic", fname)) {
    DataAsymp <- cbind(x1, DataAsymp)
  } else if (grepl("^Symptomatic", fname)) {
    DataSymp <- cbind(x1, DataSymp)
  }
}
DataSymp  <- t(DataSymp)
DataAsymp <- t(DataAsymp)
DataBoth  <- rbind(DataAsymp, DataSymp)

stopifnot(
  "Expected 8 asymptomatic subjects" = nrow(DataAsymp) == 8,
  "Expected 9 symptomatic subjects"  = nrow(DataSymp) == 9
)

# B-spline smoothing identical to funTDA.R: order-2 basis (piecewise linear),
# nbasis = Q - 1 = 499 knots over [0, 1].
create_fd <- function(Data) {
  x <- seq(0, 1, length.out = ncol(Data))
  basis <- create.bspline.basis(c(0, 1), nbasis = ncol(Data) - 1, norder = 2)
  fdobj <- smooth.basis(x, t(Data), basis)
  fdobj$fd
}

fdSymp  <- create_fd(DataSymp)
fdAsymp <- create_fd(DataAsymp)
fdBoth  <- create_fd(DataBoth)
PH_s1 <- read.csv(file.path(ph_dir, "Symptomaticadjmatrix1_PH.csv"), header = FALSE)
colnames(PH_s1) <- c("Birth", "Death", "dimension")

diagram_plot <- ggplot(PH_s1, aes(x = Birth, y = Death, shape = factor(dimension))) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", color = "grey50") +
  geom_point(size = 2, alpha = 0.7, color = col_symp) +
  scale_shape_manual(values = c("0" = 16, "1" = 17), name = "Dimension",
                     labels = c("H0", "H1")) +
  labs(x = "Birth", y = "Death", title = "Persistence diagram") +
  coord_equal() +
  theme_classic(base_size = 13) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))

ls_s1 <- r_landscape1(PH_s1, dim = 1, tseq = tseq)
landscape_plot <- ggplot(data.frame(t = tseq, lambda = ls_s1), aes(x = t, y = lambda)) +
```

```r
  geom_line(color = col_symp, linewidth = 0.9) +
  labs(x = "t", y = expression(lambda[1](t)), title = "Persistence landscape (H1)") +
  theme_classic(base_size = 13) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))

diagram_landscape_example <- diagram_plot + landscape_plot
diagram_landscape_example

# Exported for the memoria (ResearchProject_UCD/figures/diagram_landscape_example.pdf).
if (dir.exists(fig_dir)) {
  ggsave(file.path(fig_dir, "diagram_landscape_example.pdf"), diagram_landscape_example,
         width = 9, height = 4.5)
}
indiv_list <- list()
for (i in seq_len(nrow(DataAsymp))) {
  indiv_list[[length(indiv_list) + 1]] <- data.frame(
    t = tseq, lambda = DataAsymp[i, ], subject = paste0("A", i), Group = "Asymptomatic"
  )
}
for (i in seq_len(nrow(DataSymp))) {
  indiv_list[[length(indiv_list) + 1]] <- data.frame(
    t = tseq, lambda = DataSymp[i, ], subject = paste0("S", i), Group = "Symptomatic"
  )
}
indiv_df <- do.call(rbind, indiv_list)
y_max <- max(indiv_df$lambda) * 1.05

plot_group_landscapes <- function(df, group_label, col) {
  ggplot(df, aes(x = t, y = lambda, group = subject)) +
    geom_line(color = col, alpha = 0.6, linewidth = 0.6) +
    scale_y_continuous(limits = c(0, y_max)) +
    labs(x = "t", y = expression(lambda[1](t)), title = group_label) +
    theme_classic(base_size = 13) +
    theme(plot.title = element_text(hjust = 0.5, face = "bold"))
}

plot_asymp <- plot_group_landscapes(indiv_df[indiv_df$Group == "Asymptomatic", ], "Asymptomatic
plot_symp  <- plot_group_landscapes(indiv_df[indiv_df$Group == "Symptomatic", ], "Symptomatic"

group_landscape_comparison <- plot_asymp + plot_symp
group_landscape_comparison

# Exported for the memoria (ResearchProject_UCD/figures/group_landscape_comparison.pdf).
if (dir.exists(fig_dir)) {
  ggsave(file.path(fig_dir, "group_landscape_comparison.pdf"), group_landscape_comparison,
         width = 6.5, height = 4.5)
}
```

```r
pca <- pca.fd(fdBoth, nharm = 2)

pc1_pct <- pca$varprop[1] * 100
pc2_pct <- pca$varprop[2] * 100
cum_pct <- pc1_pct + pc2_pct

m <- c("A1", "A2", "A3", "A4", "A5", "A6", "A7", "A8",
       "S1", "S2", "S3", "S4", "S5", "S6", "S7", "S8", "S9")
group <- c(rep("Asymptomatic", 8), rep("Symptomatic", 9))

pca_df <- data.frame(
  PC1 = pca$scores[, 1],
  PC2 = pca$scores[, 2],
  ID = m,
  Group = group
)

n_asymp_pos <- sum(pca_df$PC1[pca_df$Group == "Asymptomatic"] > 0)
n_symp_neg  <- sum(pca_df$PC1[pca_df$Group == "Symptomatic"] < 0)
knitr::kable(
  data.frame(
    Component = c("PC1", "PC2", "Cumulative (PC1+PC2)"),
    `% variance explained` = c(sprintf("%.2f%%", pc1_pct), sprintf("%.2f%%", pc2_pct), sprintf
    check.names = FALSE
  ),
  align = "lr"
)
fpca_score_plot <- ggplot(pca_df, aes(x = PC1, y = PC2, color = Group, label = ID)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "black") +
  geom_point(size = 4, alpha = 0.9) +
  ggrepel::geom_text_repel(size = 4, show.legend = FALSE, max.overlaps = Inf) +
  scale_color_manual(values = c("Asymptomatic" = col_asymp, "Symptomatic" = col_symp)) +
  labs(x = "PC1 Score", y = "PC2 Score", title = "") +
  coord_fixed() +
  theme_classic(base_size = 16) +
  theme(
    legend.position = "right",
    legend.title = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold")
  )
fpca_score_plot

# Exported for the memoria (ResearchProject_UCD/figures/fpca_scores.pdf).
if (dir.exists(fig_dir)) {
  ggsave(file.path(fig_dir, "fpca_scores.pdf"), fpca_score_plot,
         width = 6.5, height = 4.5)
```

```
}
pca_full_df <- data.frame(
  ID = pca_df$ID,
  Group = pca_df$Group,
  PC1 = round(pca_df$PC1, 4),
  PC2 = round(pca_df$PC2, 4)
)
knitr::kable(pca_full_df, align = "llrr")
tol_fpca <- 0.1  # percentage points
stopifnot(
  "PC1 out of tolerance relative to baseline" = abs(pc1_pct - ref$pc1_pct) < tol_fpca,
  "PC2 out of tolerance relative to baseline" = abs(pc2_pct - ref$pc2_pct) < tol_fpca,
  "Expected all 8 asymptomatic subjects with PC1 > 0" = n_asymp_pos == 8,
  "Expected exactly 8 of 9 symptomatic subjects with PC1 < 0" = n_symp_neg == 8
)
rowVar <- function(M) {
  n <- ncol(M)
  mu <- rowMeans(M)
  rowSums((M - mu)^2) / (n - 1)
}

vtperm <- function(X, i1, i2, nperm = 10000) {
  x1 <- X[, i1, drop = FALSE]
  x2 <- X[, i2, drop = FALSE]
  n1 <- ncol(x1); n2 <- ncol(x2)
  Xmat <- cbind(x1, x2)
  n <- n1 + n2
  Ts <- function(M) {
    m1 <- rowMeans(M[, 1:n1, drop = FALSE])
    m2 <- rowMeans(M[, n1 + (1:n2), drop = FALSE])
    v1 <- rowVar(M[, 1:n1, drop = FALSE]) / n1
    v2 <- rowVar(M[, n1 + (1:n2), drop = FALSE]) / n2
    max(abs(m1 - m2) / sqrt(v1 + v2))
  }
  Tobs <- Ts(Xmat)
  Tn <- numeric(nperm)
  for (i in 1:nperm) Tn[i] <- Ts(Xmat[, sample(n)])
  list(pval = mean(Tobs < Tn), Tobs = Tobs, crit95 = unname(quantile(Tn, 0.95)))
}
argvals <- seq(0, 1, length.out = 101)
cache_file <- "../output/results_partA_permtest_cache.rds"

# Cache keyed on a hash of its own input CSVs (same pattern as Part B's
# run_permtest_suite): if the committed PH diagrams or adjacency matrices
# ever change, the stored p-values would otherwise silently go stale and
# asserts would pass against outdated numbers instead of catching the change.
# method = "radix" sorts in the C locale whatever LC_COLLATE says. Plain sort()
```

```r
# is locale-dependent and orders Symptomaticadjmatrix1_PH.csv before
# ...matrix10_PH.csv here, where the C locale does the opposite ("0" < "_").
# The hash is a concatenation in file order, so without this it would differ
# between machines, the cache would look invalid on any clone, and the report
# would recompute 197 permutation tests to arrive at the same numbers.
input_files <- sort(Sys.glob(c("../output/PH/*.csv", "../data/GRN_adjacency_matrices/*.csv")),
                    method = "radix")
input_hash  <- paste(unname(tools::md5sum(input_files)), collapse = "_")
cached      <- if (file.exists(cache_file)) readRDS(cache_file) else NULL
cache_valid <- !is.null(cached) && identical(cached$input_hash, input_hash)

if (!cache_valid) {
  set.seed(123)

  # 1) Main test: Asymptomatic vs Symptomatic
  XFull <- cbind(eval.fd(argvals, fdAsymp), eval.fd(argvals, fdSymp))
  test1 <- vtperm(XFull, 1:8, 9:17, nperm = 10000)

  # 2) Symptomatic split-half: combn(1:9, 4) = 126 4-vs-5 partitions
  XSymp <- eval.fd(argvals, fdSymp)
  splitsS <- combn(1:9, 4)
  pvalsS <- numeric(ncol(splitsS))
  for (i in seq_len(ncol(splitsS))) {
    g1 <- splitsS[, i]; g2 <- setdiff(1:9, g1)
    pvalsS[i] <- vtperm(XSymp, g1, g2, nperm = 10000)$pval
  }

  # 3) Asymptomatic split-half: combn(1:8, 4) = 70 4-vs-4 partitions.
  # Because the complement of a 4-of-8 subset is also a 4-of-8 subset, every
  # unique 4-vs-4 partition appears twice among these 70 (once with each
  # half as group1) - 35 unique partitions, each tested twice. Not
  # collapsed, to keep reproducing the baseline. The symptomatic loop above
  # (combn(1:9, 4), 4-vs-5) does not have this issue: a 5-element complement
  # is never itself drawn as a 4-element subset, so all 126 are distinct.
  XAsymp <- eval.fd(argvals, fdAsymp)
  splitsA <- combn(1:8, 4)
  pvalsA <- numeric(ncol(splitsA))
  for (i in seq_len(ncol(splitsA))) {
    g1 <- splitsA[, i]; g2 <- setdiff(1:8, g1)
    pvalsA[i] <- vtperm(XAsymp, g1, g2, nperm = 10000)$pval
  }

  cached <- list(input_hash = input_hash, test1 = test1, pvalsS = pvalsS, pvalsA = pvalsA)
  saveRDS(cached, cache_file)
}

test1  <- cached$test1
```

```r
pvalsS <- cached$pvalsS
pvalsA <- cached$pvalsA

stopifnot(
  "Expected 126 symptomatic splits (combn(9,4))" = length(pvalsS) == 126,
  "Expected 70 asymptomatic splits (combn(8,4))" = length(pvalsA) == 70
)
tol_p <- 0.002
tol_T <- 0.05
stopifnot(
  "Main test p-value out of tolerance" = abs(test1$pval - ref$test1_p) < tol_p,
  "Observed Tmax out of tolerance" = abs(test1$Tobs - ref$test1_Tobs) < tol_T,
  "95% critical value out of tolerance" = abs(test1$crit95 - ref$test1_crit95) < tol_T
)
splithalf_summary <- data.frame(
  Group = c("Symptomatic (126 splits, 4 vs 5)", "Asymptomatic (70 splits, 4 vs 4)"),
  `mean p-value` = sprintf("%.3f", c(mean(pvalsS), mean(pvalsA))),
  `sd(p-value)` = sprintf("%.3f", c(sd(pvalsS), sd(pvalsA))),
  `% splits with p < 0.05` = sprintf("%.1f%%", c(mean(pvalsS < 0.05), mean(pvalsA < 0.05)) * 10
  check.names = FALSE
)
knitr::kable(splithalf_summary, align = "lrrr")
histS <- ggplot(data.frame(p = pvalsS), aes(x = p)) +
  geom_histogram(binwidth = 0.05, boundary = 0, fill = col_symp, alpha = 0.6, color = "white")
  geom_vline(xintercept = 0.05, linetype = "dashed", color = "black") +
  labs(title = "Symptomatic (126 splits)", x = "p-value", y = "Frequency") +
  theme_classic(base_size = 13)

histA <- ggplot(data.frame(p = pvalsA), aes(x = p)) +
  geom_histogram(binwidth = 0.05, boundary = 0, fill = col_asymp, alpha = 0.6, color = "white"
  geom_vline(xintercept = 0.05, linetype = "dashed", color = "black") +
  labs(title = "Asymptomatic (70 splits)", x = "p-value", y = "Frequency") +
  theme_classic(base_size = 13)

splithalf_hist_combined <- histA + histS
splithalf_hist_combined

# Exported for the memoria (ResearchProject_UCD/figures/splithalf_hist.pdf).
if (dir.exists(fig_dir)) {
  ggsave(file.path(fig_dir, "splithalf_hist.pdf"), splithalf_hist_combined,
         width = 6.5, height = 4.5)
}
tol_mean <- 0.02
tol_sd   <- 0.02
tol_frac <- 2.0  # percentage points
stopifnot(
  "Symptomatic mean p out of tolerance"  = abs(mean(pvalsS) - ref$symp_mean) < tol_mean,
```

```
  "Symptomatic sd p out of tolerance"    = abs(sd(pvalsS) - ref$symp_sd) < tol_sd,
  "Symptomatic frac<0.05 out of tolerance" = abs(mean(pvalsS < 0.05) * 100 - ref$symp_frac05) <
  "Asymptomatic mean p out of tolerance" = abs(mean(pvalsA) - ref$asymp_mean) < tol_mean,
  "Asymptomatic sd p out of tolerance"   = abs(sd(pvalsA) - ref$asymp_sd) < tol_sd,
  "Asymptomatic frac<0.05 out of tolerance" = abs(mean(pvalsA < 0.05) * 100 - ref$asymp_frac05)
)
library(TDA)
tseq <- seq(0, 1, length.out = 500)
maxdiff <- 0
for (f in ph_files) {
  PH <- read.csv(f, header = FALSE)
  colnames(PH) <- c("Birth", "Death", "dimension")
  PHm <- as.matrix(PH[c("dimension", "Birth", "Death")])
  ref_landscape  <- landscape(PHm, dimension = 1, KK = 1, tseq)
  mine_landscape <- r_landscape1(PH, dim = 1, tseq = tseq)
  maxdiff <- max(maxdiff, max(abs(ref_landscape - mine_landscape)))
}
maxdiff  # 0 across the 17 subjects
set.seed(123)
ref_test  <- tperm.fd(fdAsymp, fdSymp, nperm = 500, q = 0.05, plotres = FALSE)
set.seed(123)
mine_test <- vtperm(cbind(eval.fd(argvals, fdAsymp), eval.fd(argvals, fdSymp)),
                    1:8, 9:17, nperm = 500)
isTRUE(all.equal(ref_test$pval, mine_test$pval)) # TRUE
isTRUE(all.equal(ref_test$Tobs, mine_test$Tobs)) # TRUE
sessionInfo()
```

Session information

```
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: aarch64-apple-darwin20
Running under: macOS Tahoe 26.5.2

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LA

locale:
[1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8

time zone: Europe/Madrid
tzcode source: internal
```

```
attached base packages:
[1] splines   stats     graphics  grDevices utils     datasets  methods
[8] base

other attached packages:
 [1] igraph_2.3.2    patchwork_1.3.2 ggrepel_0.9.8   ggplot2_4.0.3
 [5] fda_6.3.0       deSolve_1.42    fds_1.9         RCurl_1.98-1.19
 [9] rainbow_3.8     pcaPP_2.0-5     MASS_7.3-65

loaded via a namespace (and not attached):
 [1] ks_1.15.2          generics_0.1.4     bitops_1.0-9       KernSmooth_2.23-26
 [5] lattice_0.22-9     hdrcde_3.5.0       pracma_2.4.6       digest_0.6.39
 [9] magrittr_2.0.5     evaluate_1.0.5     grid_4.5.3         RColorBrewer_1.1-3
[13] mvtnorm_1.4-2      fastmap_1.2.0      jsonlite_2.0.0     Matrix_1.7-4
[17] mclust_6.1.3       scales_1.4.0       cli_3.6.6          rlang_1.2.0
[21] withr_3.0.2        yaml_2.3.12        tools_4.5.3        dplyr_1.2.1
[25] colorspace_2.1-3   vctrs_0.7.3        R6_2.6.1           lifecycle_1.0.5
[29] cluster_2.1.8.2    pkgconfig_2.0.3    pillar_1.11.1      gtable_0.3.6
[33] glue_1.8.1         Rcpp_1.1.1         xfun_0.57          tibble_3.3.1
[37] tidyselect_1.2.1   knitr_1.51         farver_2.1.2       htmltools_0.5.9
[41] rmarkdown_2.31     labeling_0.4.3     compiler_4.5.3     S7_0.2.2
```